ISyE 6740 – Spring 2021
Final Report

Team Member Name: Alvaro Esteban Larreategui, GT Id *****0323

Project Title: Analysis of Histopathological Images for Disease Detection

Introduction

"Contrary to popular belief, the first forays into the use of digital image processing and computerized image analysis were not in face recognition or face detection, but rather for the analysis of cell and microscopy images" [(1)].

Histopathology has for a long time been a human-centered task: pathologists visually analyze tissue samples under microscope to detect features and patterns related to disease, particularly cancer. The whole process is time consuming, first because of the preparation of the tissue, the staining with -for example- H&E, and finally the visual inspection.

Digital pathology was born in the 1980s, with important limitations like slow-scanning speed, low memory capacity and short network bandwidth. Since then, developments in digitation of images have facilitated digital storing and sending of digital histopathology images. Nowadays, WSI (Whole Slide Images) allow high quality images.

However, there are still opportunities to render the process more efficient. For example, 80% of the time of the pathologist is spent visualizing benign cells. In recent years ML methods have been developed to help also in the analysis of the images, particularly through CAD: Computer assisted Diagnosis.

Data Sources

The dataset used for this project is the PatchCamelyon image classification dataset.
It consists of 327.680 color images (96 x 96px) extracted from histopathologic scans of lymph node sections. The center 32x32 square of each patch contains at least 1 pixel of metastatic tissue. Both training and test sets are provided, with labels: 1 for presence of metastasis and a 0 for the absence of metastatic tissue.

Link to data: https://github.com/basveeling/pcam

Computational Tools

Coding was performed using Python 3.11 in Jupyter Notebook, and mainly with ML tools from the **sci-kit learn** module.

Problem Statement

For this project I consider the problem of detecting the presence of cancer in a tissue using histopathology images. This problem can be viewed naturally as a classification problem (classifying the samples into presence/absence of cancer). But I wonder whether applying techniques such as non-linear dimensionality reduction as a way of identifying non-linear patterns in tissue can improve the classification.

Research Questions

1. Pathologists use their domain expertise to detect whether a tissue has a presence of cancer. Can we train a classification model in a way to detect whether cancer is present in a tissue sample?

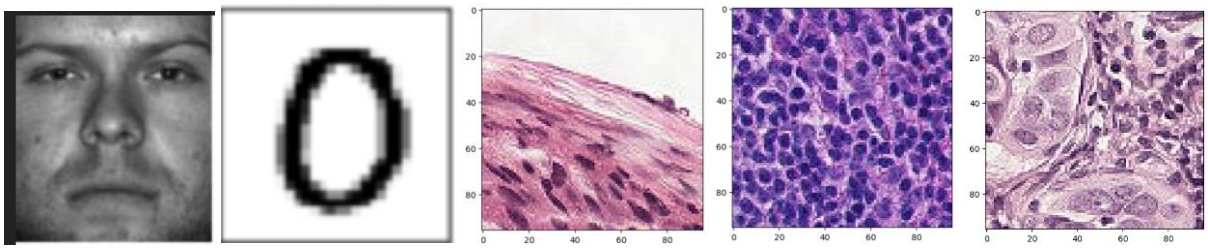
2. Pathologists identify structures such as nuclei and classify tissue mainly based on architectural patterns. Can non-linear dimensionality reduction be used to extract cell architecture features which are inherently non-linear, to improve detection?

A note on Histopathological Images

As soon as I started to craft a project strategy, I detected a difference with other known ML datasets such as MNIST or Yale faces that is worth commenting on.

MNIST or Yale faces usually have the object (digit or face) well centered in the image frame. This implies that when the image is vectorized, each pixel is treated as a feature that represents a certain characteristic of the object. Think of this as for example "Income" in an economic dataset. Suppose for example, that digits are stored in 5x5-pixel image, all well centered and occupying roughly the same proportion. Then the pixel at the center would be the 13^{th} component of the feature vector, which has the characteristic of being roughly the center of all objects. As such, it would likely be "black" in the case of a digit 1 or "white" in the case of the digit 0. But histopathological images do not show a well-defined object, because they show tissue: an irregular array of cells, membranes, etc. Would this affect the quality of the classification?

Fig.1: Yalefaces and MNIST vs PCAM images



Research Question 1: **Can we train a classification model to detect cancer in a tissue sample?**

Analytical Approach

Since the data provides a label, the classification setting is pretty much straightforward. I carried out different combinations of classifiers, train-test split sizes and source images as detailed below:

- Classifiers:
  The classifiers were chosen based on the inherent non-linearity of the data, and the binary nature of the classification. Not all of them were used at each stage or phase. For example, SVM was included basically out of curiosity, even though it was expected to perform poorly. For the first 3 phases, the following classifiers were used:

  - **KNeighborsClassifier** with k=5 in all of the Phases, including CV
  - **LogisticRegression** with default parameters
  - **SVC** with 'linear' kernel and KSVM with 'rbf' kernel and C=5
  - **MLPClassifier** with hidden layers (10,10)
  - **AdaBoostClassifier**(DecisionTreeClassifier()) with n_estimators=200

  Table 1 provides a detail of each scenario. At each phase, the selection was adjusted based on previous results. At the end of the 3rd phase, the best performing models were KNN, KSVM and AdaBoost. Thus for phase 4 I did a fine tuning of these 3 classifiers using **RandomizedSearchCV** and **GridSearchCV**, which resulting in the following hyperparameters for each:

  - KNN with k=5
  - KSVM with 'rbf' kernel, C=5 and gamma = 0.5
  - AdaBoost with 100 estimators and max_depth=10

- Images:
  Original full 96x96 image patches were used in the first stage, but runtimes were very high for some classifiers compared to others. Thus, because the 32 x 32 pixels center contained the information for classification, and the outer "frame" was mostly used for consistency when using Convolutional Neural Networks, decision was made to use only the center for the remaining scenarios.

- Color channels:
  RGB color channels (stacked) were used in the first two stages. This resulted in 27,648 and 3,072 features respectively. Presumably this impacted the runtimes for a couple of classifiers, so I decided to use grayscale instead for the last two phases.

Table 1: classification phases

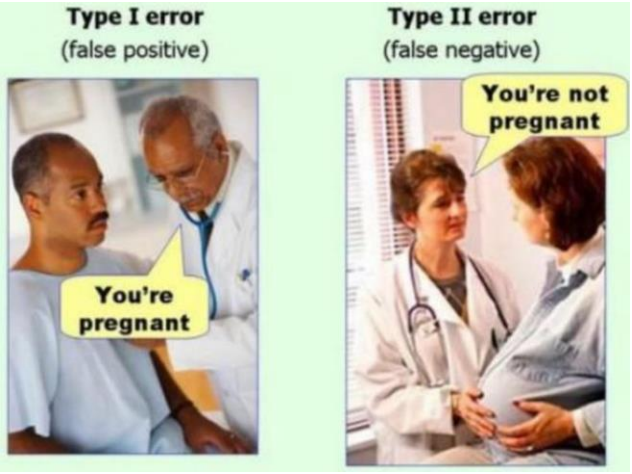
						Classifiers					
Phase	Picture Size	Color Channels	Number of Features	Train Size	Test Size	KNN	Logistic Regression	SVM	kSVM	Neural Net	AdaBoost
1	96 x 96	RGB	27,648	5,000	1,000	*	*	*	*		
2	32 x 32	RGB	3,072	10,000	2,000	*	*	*	*	*	*
3	32 x 32	Grayscale	1,024	1,000	500	*	*		*	*	*
4	32 x 32	Grayscale	1,024	5,000	5,000	**			**		**

* *Arbitrary parameters*
** *Parameters fine-tuned via CVSearch or SeacrhGridCV*

Evaluation & Results

As in any classification problem, the confusion matrix was used to obtain a crosstabulation of the predicted class vs the true class or label. It is important to note that in this problem the cost of misclassifying a true case of disease (false negative or Type II error) is much higher than the cost of misclassifying a negative case (false positive or Type I error). With a false negative the life of the patient is put at risk, whereas the false negative rarely anything beyond emotional distress might be caused. Figure 2 is a useful mnemonic depiction of these two types of errors.

Fig.2: misclassification errors



In consequence, the primary measure that we use for evaluation is the type II error, which we want to minimize. Or conversely, we want to maximize the Sensitivity. In confusion matrix jargon, the type II error is also called 1 – sensitivity. A secondary and complementary measure that is used here is the Specificity, which the probability of a true negative, or in other words it is 1 – Type I error.

Remember that the PCAM dataset employs the label 0 for absence of cancer in the patch and 1 for presence of cancer in the patch. Accordingly, we will call 0 the negative case and 1 the true positive case. This way $y = 0$ will represent true Negative, $\hat{y} = 0$ will represent predicted negative, and similarly for the positive cases.

Table 2 shows the sensitivity and Table 3 the specificity for the classifiers in each of the scenarios. In general, is noticeable a trade-off between sensitivity and specificity.

Table 2: Sensitivity achieved by each classifier.

	Sensitivity					
Phase	KNN	Logistic Regression	SVM	kSVM	Neural Net	AdaBoost
1	91.3%	63.6%	61.0%	73.0%		
2	89.1%	60.6%	60.9%	71.9%	63.0%	74.1%
3	94.6%	59.3%		76.8%	49.8%	75.5%
4	91.1%			99.4%		77.2%

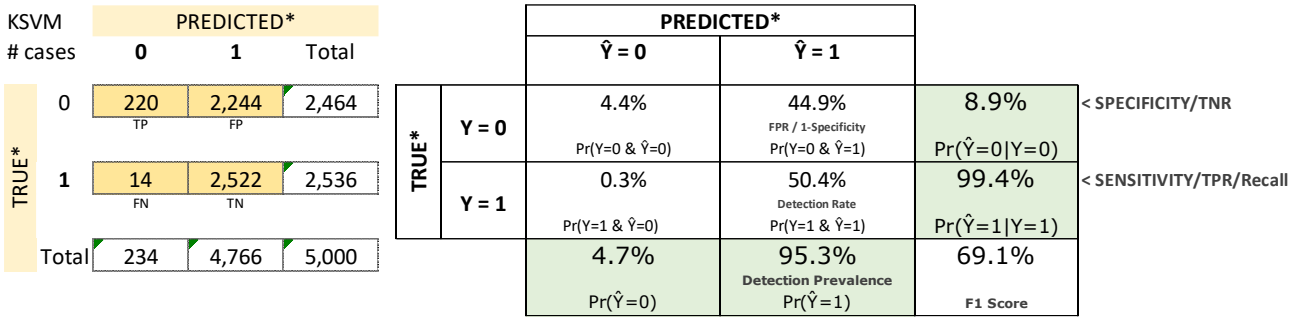
Table 3: Specificity achieved by each classifier.

	Specificity					
Phase	KNN	Logistic Regression	SVM	kSVM	Neural Net	AdaBoost
1	42.2%	60.0%	64.6%	77.2%		
2	42.8%	69.1%	66.4%	76.3%	63.5%	70.6%
3	37.5%	61.0%		69.1%	66.4%	61.8%
4	38.6%			8.9%		64.0%

Even though it was established that sensitivity was the primary metric, one must consider a balance between both sensitivity and specificity. Thus, the best performing model is KNN.

Let's explain why KSVM is not selected as best. Figure 3 shows the confusion matrix for KSVM in phase 4, showing a few of the measures derived from it, with their different names.

Fig.3: confusion matrix and related metric for KSVM, phase 4



Here, 99.4% sensitivity comes from 2,522 out of 2,536 true disease cases classified correctly. So far, so good, but consider that only 220 out of 2,464 healthy tissues were classified correctly (resulting in 8.9% specificity). This situation is close to classifying as "healthy" all the samples, thus reaching 100% sensitivity, 0% specificity. This is not what we want from a binary classifier.

It is also worth mentioning a little about the runtimes of each fitting - predicting cycle. Considering the computing environment mentioned at the beginning, KNN was amazingly fast, performing consistently under 1 minute, for all image sizes!

KSVM on the other hand, was fast only with reduced patches. Adaboost was also rather slow, running in the order of several minutes. Table 4 presents the runtimes for each classifier at each stage or phase.

Table 4: classifier runtimes

	Training & Prediction runtime (minutes)					
Phase	KNN	Logistic Regression	SVM	kSVM	Neural Net	AdaBoost
1	0.1	1.0	12.0	15.0		
2	0.3	0.5	25.0	4.0	0.5	5.0
3	0.1	0.0		0.0	0.0	2.5
4	0.1			0.5		23.1

Research Question 2: **Can non-linear dimensionality reduction be used to extract cell architecture features which are inherently non-linear, to improve detection?**

Analytical Approach

As mentioned in the initial comments, when an image is vectorized, each pixel is considered a feature and its relative position within the feature vector is relevant in the classification problem. This approach works well in famous cases such as MNIST or Face Detection, where the digits or faces are centered within the image. But this is not true for PCAM images, which represent tissue, cells, membranes, and other structures. Thus, the relative position of a given pixel in one patch would a priori have nothing to do with the same pixel in another patch. This leads me to think that dimensionality reduction procedures would not impact greatly the information about the position of the pixel with respect to the image. And it could lead to improvements in runtimes.

In the previous section, the classifiers were evaluated using either RGB or grayscale input images, which meant 3,072 and 1,024 features respectively. What if we can reduce the dimensionality to a few features only, while keeping the information as much as possible, and then use them as input for the classifiers? In this case, I consider non-linear manifold learning techniques such as ISOMAP that was seen in class, to reduce the dimensionality.

The process began with the central 32x32 patches being transformed to grayscale. Then those were used as input for 4 different manifold learning techniques:

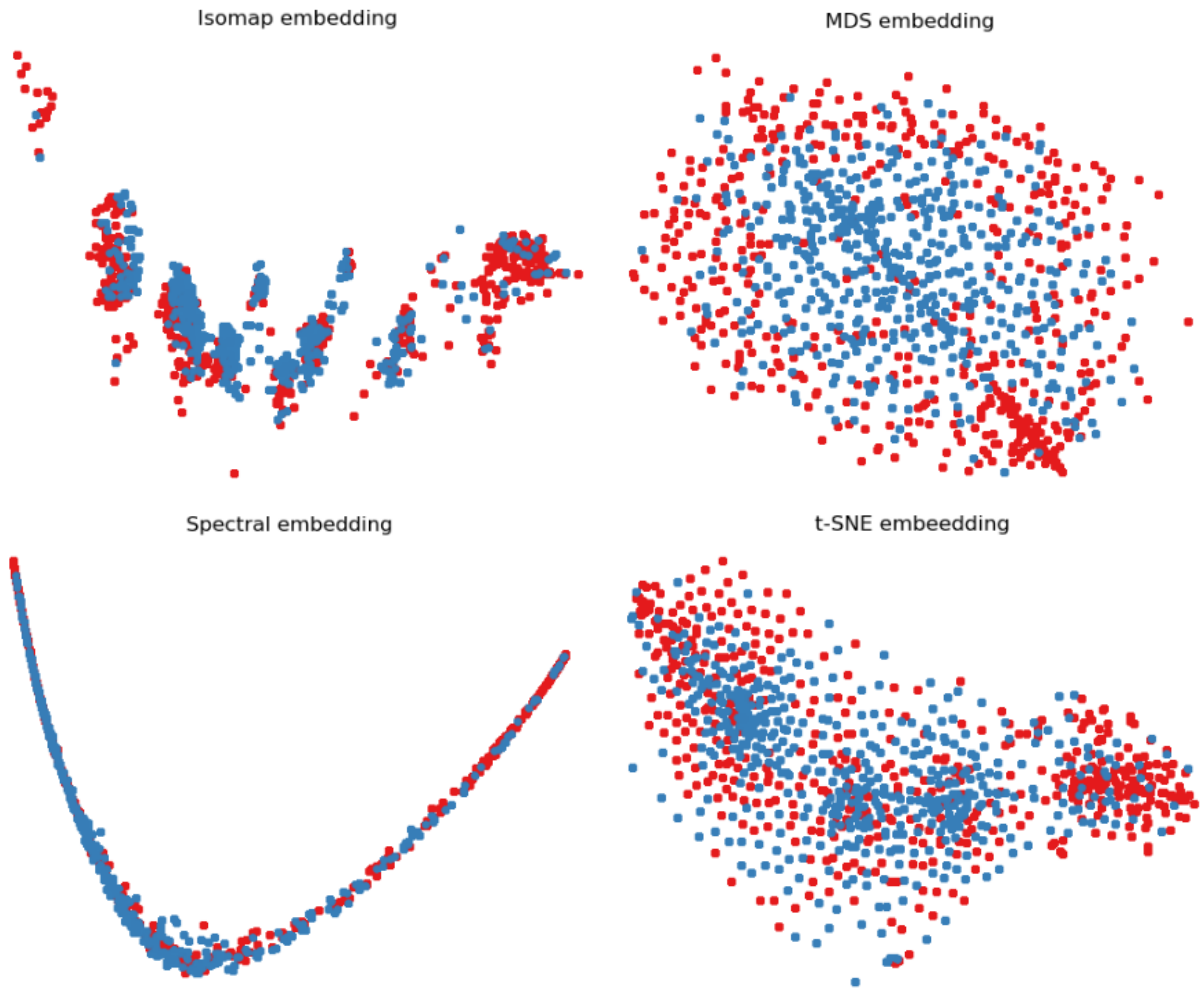
- Isomap
- Multi-Dimensional Scaling (MDS)
- Spectral Embedding
- t-SNE

The code I used was based on the one provided by scikit learn manifold learning page. (https://scikit-learn.org/stable/auto_examples/manifold/plot_lle_digits.html#sphx-glr-auto-examples-manifold-plot-lle-digits-py)

The resulting embeddings from a subset of the training data are depicted in Figure 4, where the two classes are represented by two different colors. Apparently, Isomap is the one that best separates the classes in such a way that either KNN or KSVM could perform well. Thus, ISOMAP was chosen to fit the classifiers.

Thus, I applied ISOMAP to reduce the dimensionality to different number of features, specifically: 5, 25, 50, 100 and 500. However, since this affected greatly the structure in the data, a fine tuning via **GridSearchCV** and **RandmoizedSearchCV** was again performed to find the best parameters for fitting the two best performing classifiers from the previous section (KNN and KSVM).

Fig. 4: manifold learning embeddings



Evaluation & Results

The Nonlinear dimensionality reduction applied as pre-processing did not really improve the classification results. The best results were achieved by KNN consistently across the different scenarios, with a sensitivity between 70% and 79% and specificity between 50% and 59%. But these results were inferior to those achieved without dimensionality reduction. Table 5 presents the results for each selected number of components.

Table 5: classification results on reduced number of components

# components		KNN	KSVM
5	hyper-parameters	k=17	C = 3 gamma = 0.25
	sensitivity	79%	18%
	specificity	51%	93%
25	hyper-parameters	k=19	C = 2 gamma = 0.1
	sensitivity	73%	31%
	specificity	57%	85%
50	hyper-parameters	k=15	C = 2 gamma = 0.25
	sensitivity	72%	99%
	specificity	58%	1%
100	hyper-parameters	k=12	C = 2 gamma = 0.25
	sensitivity	69%	99.6%
	specificity	59%	0%
500	hyper-parameters	k=11	C = 1 gamma = 0.05
	sensitivity	69%	95.4%
	specificity	58%	21%

On the other hand, KSVM achieved highly imbalanced results, for example in the cases with 50 and 100 components, it achieved a sensitivity of 99%, coupled with a specificity of 1% or 0%.

Figure 5: confusion matrix results for #100 ISOMAP components

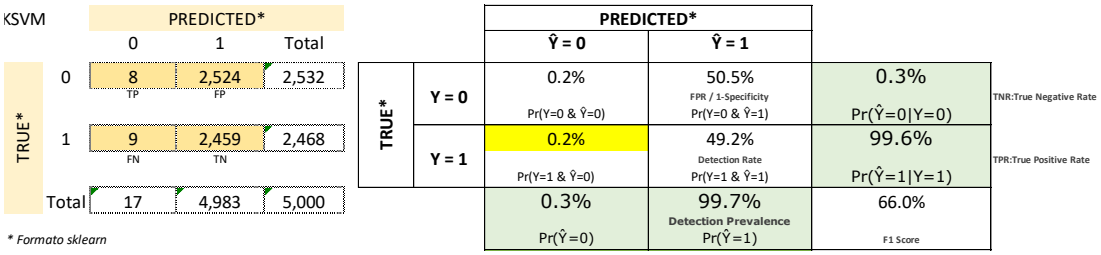


Figure 5 shows the crosstabulation as well as the probabilities associated with the classification from KSVM of the 100 ISOMAP components: very few cases are predicted as 0 overall. This is not suitable, because there is little difference compared to assigning a label 1 to all cases in the test data!

Unfortunately, as it has been shown, ISOMAP did not improve classification results.

Conclusion

Concerning our initial research questions, both were answered with the PCAM dataset:

1. Can we train a classification model to detect cancer in a tissue sample?

   Yes, it was possible to fit a few classifiers that performed relatively well, the best being a KNN classifier that achieved 91.1% sensitivity, 38.6% specificity on a test dataset of 5,000 image patches of size 32x32 pixels (1,024 features)

2. Can non-linear dimensionality reduction be used to extract cell architecture features which are inherently non-linear, to improve cancer detection?

   No, it was not possible to improve the best model found previously. At least using ISOMAP projection and different number of components (5, 25, 50, 100, 500).

As a personal note, I found KNN such a simple, fast and yet powerful algorithm. It is just remarkable!

Concerning our additional question on the non-symmetrical nature of histopathological images, presumably this did not affect the classification, given the results obtained. Further analysis could be performed by selecting more similar, center and/or symmetrical images.

It was very interesting working with the PCAM image dataset. With more than 300,000 images to play with, there is room for more exploration and potential improvements. With more time, the following avenues could have been explored:

- Additional color preprocessing techniques beyond grayscale averaging (e.g. color filters…)
- A better design/pipeline for evaluating classifiers and study the effects of train-test split sizes,
- Manifold learning techniques other than ISOMAP
- PCA to reduce dimensionality.

References

1. Image analysis and machine learning in digital pathology: Challenges and opportunities.
https://www.sciencedirect.com/science/article/pii/S1361841516301141?via%3Dihub

2. Chapter 10 - Analysis of Histopathology Images: From Traditional Machine Learning to Deep Learning.
https://www.sciencedirect.com/science/article/pii/B9780128121337000107

3. Histopathology datasets for Machine Learning.
https://github.com/maduc7/Histopathology-Datasets#histopathology-datasets-for-machine-learning.